

АННОТАЦИЯ

Выпускная квалификационная работа на тему «Разработка веб-сервиса заказа и доставки продуктов».

В первой главе проведен анализ предметной области веб-сервисов доставки продуктов, рассмотрена классификация существующих решений, выполнен сравнительный анализ аналогов (СберМаркет, Яндекс Лавка, Самокат), обоснована необходимость разработки собственного сервиса и сформулированы требования к нему.

Во второй главе спроектирована микросервисная архитектура системы, разработаны диаграммы вариантов использования и активности, описана структура базы данных для каждого микросервиса, представлены макеты пользовательского интерфейса (главная страница, каталог, корзина, оформление заказа, личный кабинет, панель администратора), формализованы алгоритмы аутентификации, оформления заказа и управления статусами заказов.

В третьей главе описан процесс разработки веб-сервиса: выбраны технологии (Python/FastAPI, PostgreSQL, Docker, TypeScript/Next.js), реализована контейнеризация микросервисов с использованием Docker Compose, разработан API Gateway для маршрутизации запросов, приведены листинги ключевых модулей, выполнено тестирование API с использованием Swagger UI.

Результатом работы является готовый к развертыванию веб-сервис заказа и доставки продуктов, ориентированный на потребности малого и среднего бизнеса.

Ключевые слова: веб-сервис, доставка продуктов, микросервисная архитектура, FastAPI, Docker, PostgreSQL, Next.js, JWT-аутентификация, API Gateway.

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ.....	3
СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	6
ВВЕДЕНИЕ	7
1. Анализ предметной области веб-сервисов заказа и доставки продуктов	9
1.1 Понятие и классификация веб-сервисов заказа и доставки продуктов	9
1.2 Обзор и сравнительный анализ существующих веб-сервисов	10
1.2.1 Анализ сервиса СберМаркет	11
1.2.2 Анализ сервиса Яндекс Лавка	11
1.2.3 Анализ сервиса Самокат	12
1.2.4 Сравнение с международными аналогами	12
1.3 Обоснование необходимости разработки и постановка требований к создаваемому веб-сервису	13
2. ПРОЕКТИРОВАНИЕ ВЕБ-СЕРВИСА ЗАКАЗА И ДОСТАВКИ ПРОДУКТОВ	15
2.1 Моделирование бизнес-процессов	15
2.1.1 Диаграмма вариантов использования (Use Case Diagram)	15
2.1.2 Диаграмма активности (Activity Diagram)	17
2.2 Проектирование архитектуры веб-сервиса	18
2.2.1 Обоснование выбора микросервисной архитектуры	18
2.2.2 Состав микросервисов и их взаимодействие	18
2.3 Проектирование базы данных	19
2.3.1 Логическая модель данных	19

2.4 Проектирование пользовательского интерфейса	22
2.4.1 Описание пользовательских сценариев	27
2.5 Проектирование алгоритмов работы системы	28
2.5.1 Алгоритм регистрации и аутентификации	29
2.5.2 Алгоритм взаимодействия микросервисов при оформлении заказа	29
2.5.3 Алгоритм управления заказами (администрирование)	30
3. РАЗРАБОТКА ВЕБ-СЕРВИСА ЗАКАЗА И ДОСТАВКИ ПРОДУКТОВ	33
3.1 Выбор средств разработки	33
3.1.1 Детальное описание User Service	34
3.1.2 Детальное описание Catalog Service	34
3.1.3 Детальное описание Cart Service	35
3.1.4 Детальное описание Order Service	35
3.1.5 Детальное описание API Gateway	35
3.1.6 Обоснование выбора пользовательского интерфейса	36
3.1.7 Требования к информационной безопасности	36
3.2 Контейнеризация и оркестрация.....	37
3.3 Реализация API Gateway.....	39
3.4 Пример реализации микросервиса.....	39
3.5 Тестирование системы.....	40
3.5.1 Тест-кейсы.....	40
3.5.2 Результаты тестирования.....	41
3.5.3 Проверка работоспособности	42
3.5.4 Тестирование пользовательского интерфейса	43
ЗАКЛЮЧЕНИЕ	45

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	47
--	----

СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

<i>Сокращение</i>	<i>Расшифровка</i>
<i>API</i>	<i>Application Programming Interface — программный интерфейс приложения</i>
<i>B2C</i>	<i>Business-to-Consumer — модель электронной коммерции, ориентированная на конечного потребителя</i>
<i>CRUD</i>	<i>Create, Read, Update, Delete — базовые операции с данными</i>
<i>Docker</i>	<i>Платформа для контейнеризации приложений</i>
<i>ER-диаграмма</i>	<i>Entity-Relationship Diagram — диаграмма сущность-связь</i>
<i>FastAPI</i>	<i>Веб-фреймворк для создания API на языке Python</i>
<i>ГОСТ</i>	<i>Государственный стандарт</i>
<i>ГЭК</i>	<i>Государственная экзаменационная комиссия</i>
<i>JWT</i>	<i>JSON Web Token — токен для аутентификации в формате JSON</i>
<i>ORM</i>	<i>Object-Relational Mapping — объектно-реляционное отображение</i>
<i>PostgreSQL</i>	<i>Реляционная система управления базами данных</i>
<i>REST</i>	<i>Representational State Transfer — архитектурный стиль веб-сервисов</i>
<i>SQL</i>	<i>Structured Query Language — язык структурированных запросов</i>
<i>SQLAlchemy</i>	<i>Библиотека ORM для Python</i>
<i>UI</i>	<i>User Interface — пользовательский интерфейс</i>
<i>UML</i>	<i>Unified Modeling Language — унифицированный язык моделирования</i>
<i>СУБД</i>	<i>Система управления базами данных</i>

ВВЕДЕНИЕ

В условиях активного развития цифровой экономики и широкого внедрения информационных технологий существенно возрастает роль веб-сервисов, ориентированных на автоматизацию процессов розничной торговли и обслуживания населения. Одним из наиболее развивающихся направлений электронной коммерции в последние годы является онлайн-заказ и доставка продуктов питания. Данный сегмент демонстрирует устойчивый рост, что обусловлено изменением образа жизни населения, ускорением темпа городской жизни, развитием логистической инфраструктуры и повышением доступности сети Интернет.

Современные веб-сервисы заказа и доставки продуктов представляют собой сложные информационные системы, обеспечивающие взаимодействие между конечными пользователями, торговыми организациями, складскими комплексами и службами доставки. Такие системы функционируют в условиях высокой конкуренции и значительных нагрузок, особенно в пиковые периоды, что предъявляет повышенные требования к их надежности, производительности и удобству использования. Кроме того, веб-сервисы данного класса обрабатывают персональные данные пользователей и сопровождают финансовые операции, что обуславливает необходимость обеспечения высокого уровня информационной безопасности.

Несмотря на наличие большого количества коммерческих решений в сфере онлайн-доставки продуктов, анализ существующих сервисов показывает, что многие из них обладают ограниченной гибкостью, сложностью сопровождения и недостаточной адаптивностью к изменяющимся требованиям рынка и пользователей. Это указывает на необходимость теоретического осмысления предметной области и систематизации подходов к проектированию веб-сервисов данного класса.

Актуальность настоящей научно-исследовательской работы обусловлена необходимостью анализа текущего состояния предметной области онлайн-заказа и доставки продуктов, а также выявления ключевых проблем и ограничений существующих решений. Целью исследования является изучение отечественного и зарубежного опыта разработки веб-сервисов доставки продуктов и формирование обобщенной характеристики проблемы, которая может быть положена в основу дальнейшей проектной и разработческой деятельности в рамках выпускной квалификационной работы.

В рамках достижения поставленной цели в работе проводится обзор зарубежных и отечественных исследований, анализируются основные направления развития веб-сервисов доставки продуктов и формулируется общая характеристика проблемы исследования.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ ВЕБ-СЕРВИСОВ ЗАКАЗА И ДОСТАВКИ ПРОДУКТОВ

1.1 Понятие и классификация веб-сервисов заказа и доставки продуктов

В условиях цифровизации экономики веб-сервисы заказа и доставки продуктов стали неотъемлемой частью инфраструктуры розничной торговли. Под веб-сервисом заказа и доставки продуктов понимается информационная система, обеспечивающая удаленное взаимодействие между покупателем и продавцом (или агрегатором) с целью выбора, оплаты и получения продуктов питания [1]. Такие системы относятся к классу электронной коммерции (модель В2С) и характеризуются высокой динамикой развития, сложностью логистических алгоритмов и необходимостью обработки больших объемов данных в реальном времени.

Анализ зарубежных и отечественных исследований [2, 4] позволяет классифицировать существующие веб-сервисы по нескольким ключевым признакам, представленным в таблице 1.1.

Таблица 1.1 – Классификация веб-сервисов заказа и доставки продуктов

Признак классификации	Типы сервисов	Краткая характеристика
По бизнес-модели	Сервисы-агрегаторы	Не имеют собственных магазинов и складов, выступают посредниками между покупателем и несколькими магазинами.
	Сервисы-дропсторы (быстрая доставка)	Имеют собственную сеть «дарксторов» (складов без покупателей) и курьерскую службу, ориентированы на доставку за 15–30 минут.

Продолжение Таблицы 1.1

	<i>Сервисы ритейлеров</i>	<i>Являются цифровым каналом сбыта существующей розничной сети (например, «Азбука Вкуса»).</i>
<i>По архитектуре ПО</i>	<i>Монолитные приложения</i>	<i>Система построена как единое целое. Проще в разработке на старте, но сложнее в масштабировании и поддержке.</i>
	<i>Микросервисные приложения</i>	<i>Функциональность разделена на множество независимых сервисов (каталог, корзина, оплата, доставка). Сложнее в разработке, но легче масштабируется и адаптируется.</i>
<i>По типу интерфейса</i>	<i>Веб-версии (сайты)</i>	<i>Доступны через браузер с любого устройства.</i>
	<i>Мобильные приложения</i>	<i>Требуют установки, но предоставляют больше возможностей для использования функций смартфона (геолокация, push-уведомления).</i>

Как видно из таблицы, выбор модели и архитектуры напрямую влияет на дальнейшую разработку. Для создания гибкого и масштабируемого решения в рамках данной ВКР целесообразно ориентироваться на микросервисную архитектуру, которая позволит независимо развивать и поддерживать различные компоненты системы.

1.2 Обзор и сравнительный анализ существующих веб-сервисов

Для определения текущего состояния рынка и выявления недостатков существующих решений проведем анализ наиболее популярных веб-сервисов доставки продуктов, представленных в Российской Федерации. В качестве

объектов анализа выбраны сервисы, представляющие разные бизнес-модели: СберМаркет (агрегатор), Яндекс Лавка (даркстор/быстрая доставка) и Самокат (даркстор/быстрая доставка).

1.2.1 Анализ сервиса СберМаркет

СберМаркет является агрегатором, который работает с партнерами («ВкусВилл», «Ашан», «Перекресток» и др.).

Используемое ПО и технологии: Сайт и мобильные приложения (iOS, Android). Технический стек включает в себя Java (бэкенд), Kotlin/Swift (мобильная разработка), React (веб). Сервис активно использует машинное обучение для прогнозирования спроса и оптимизации сборки заказов.

Целевая аудитория: Широкая аудитория, от занятых профессионалов до пожилых людей, ценящих широкий ассортимент и возможность заказать продукты из разных магазинов сразу.

Плюсы: Огромный ассортимент (десятки тысяч позиций), возможность сравнения цен разных магазинов, интеграция с бонусной программой СберСпасибо.

Минусы: относительно долгая доставка (от 2 часов), сложность логистики (сборка в магазине, а не на складе), высокая нагрузка на курьерскую службу в пиковые часы.

1.2.2 Анализ сервиса Яндекс Лавка

Яндекс Лавка работает по модели быстрой доставки. Продукты хранятся на собственных «дарксторах», расположенных в радиусе небольшой доступности от клиента.

Используемое ПО и технологии: Мощная бэкенд-инфраструктура на базе собственных технологий Яндекса (Yandex Database), мобильные приложения, собственные алгоритмы маршрутизации курьеров на картах.

Целевая аудитория: Жители крупных городов, которым важна скорость (доставка за 15– 30 минут). Чаще всего это молодые люди и семьи, которые не хотят тратить время на поход в супермаркет.

Плюсы: Высокая скорость доставки, удобное приложение с понятным интерфейсом, хорошо отлаженная логистика.

Минусы: Ограниченный ассортимент по сравнению с супермаркетами (только самые популярные товары), цены могут быть выше средних, сервис доступен только в крупных городах.

1.2.3 Анализ сервиса Самокат

Самокат – прямой конкурент Яндекс Лавки, также работающий по модели дарксторов.

Используемое ПО и технологии: интенсивно разрабатывает собственные IT-решения. Известно об использовании Go (бэкенд), Flutter (кроссплатформенная мобильная разработка). Уделяют огромное внимание автоматизации процессов внутри даркстора.

Целевая аудитория: Аналогична Яндекс Лавке, но с акцентом на более демократичную ценовую политику и собственные торговые марки (СТМ).

Плюсы: очень быстрая доставка (часто быстрее заявленного времени), удобное приложение с игровыми механиками (прогресс-бар заказа), доступные цены на базовые товары и СТМ.

Минусы: Такой же ограниченный ассортимент, как и у Яндекс Лавки, зависимость от плотности расположения дарксторов.

1.2.4 Сравнение с международными аналогами

Помимо российских сервисов, были рассмотрены международные аналоги: Instacart (США), Deliveroo (Великобритания) и Glovo (Испания).

Instacart – крупнейший сервис в США, работает по модели агрегатора. Сотрудничает с сетями Costco, Kroger, Walmart. Доставка от 1 часа, комиссия до 15% от заказа. Недоступен за пределами США и Канады.

Deliveroo – британский сервис, специализируется на доставке из ресторанов, но также доставляет продукты из магазинов. Доставка за 30-40 минут. Работает в Европе, Азии и на Ближнем Востоке.

Glovo – испанский сервис, доставляет продукты, лекарства, документы. Работает по модели агрегатора с курьерами на велосипедах и скутерах. Присутствует в 20 странах, включая Россию (Glovo прекратил работу в 2022 году).

Сравнение показывает, что международные сервисы также не предоставляют готовых решений для локального развертывания. Разработанный сервис ориентирован на малый бизнес, который не может интегрироваться с международными агрегаторами по финансовым или юридическим причинам.

1.3 Обоснование необходимости разработки и постановка требований к создаваемому веб-сервису

Проведенный анализ аналогов показал, что на рынке сложилось два доминирующих подхода: «широкий ассортимент, но долгая доставка» (СберМаркет) и «быстрая доставка, но малый ассортимент» (Яндекс Лавка, Самокат). Кроме того, все рассмотренные сервисы являются коммерческими и имеют закрытый код, что исключает возможность их адаптации под нужды малого бизнеса или конкретного региона с ограниченной логистической инфраструктурой.

Проблема исследования заключается в отсутствии гибкого, масштабируемого и экономически доступного решения для локальных

игроков, которое позволило бы им организовать собственный сервис доставки, комбинируя преимущества разных моделей.

Для решения этой проблемы необходимо разработать веб-сервис, который будет:

1. Модульным: иметь возможность подключать как собственные склады, так и магазины-партнеры.
2. Масштабируемым: выдерживать рост числа пользователей без потери производительности.
3. Функциональным: включать базовый, но достаточный набор функций (каталог, корзина, оплата, отслеживание заказа) для быстрого старта бизнеса.

Таблица 1.2 – Сравнительный анализ аналогов и разрабатываемого сервиса

Критерий сравнения	СберМаркет	Яндекс Лавка	Самокат	Разрабатываемый сервис
Бизнес-модель	Агрегатор	Даркстор	Даркстор	Гибридная (модульная)
Ассортимент	Очень широкий	Ограниченный	Ограниченный	Настраиваемый под заказчика
Скорость доставки	Низкая (часы)	Высокая (минуты)	Высокая (минуты)	Средняя/Высокая (в зависимости от выбранной модели)
Масштабируемость решения	Зависит от монополии	Требуется больших вложений	Требуется больших вложений	Высокая, за счет микросервисной архитектуры
Стоимость внедрения (для бизнеса)	Комиссия за заказ	Недоступно	Недоступно	Относительно низкая (готовое, настраиваемое решение)

2. ПРОЕКТИРОВАНИЕ ВЕБ-СЕРВИСА ЗАКАЗА И ДОСТАВКИ ПРОДУКТОВ

2.1 Моделирование бизнес-процессов

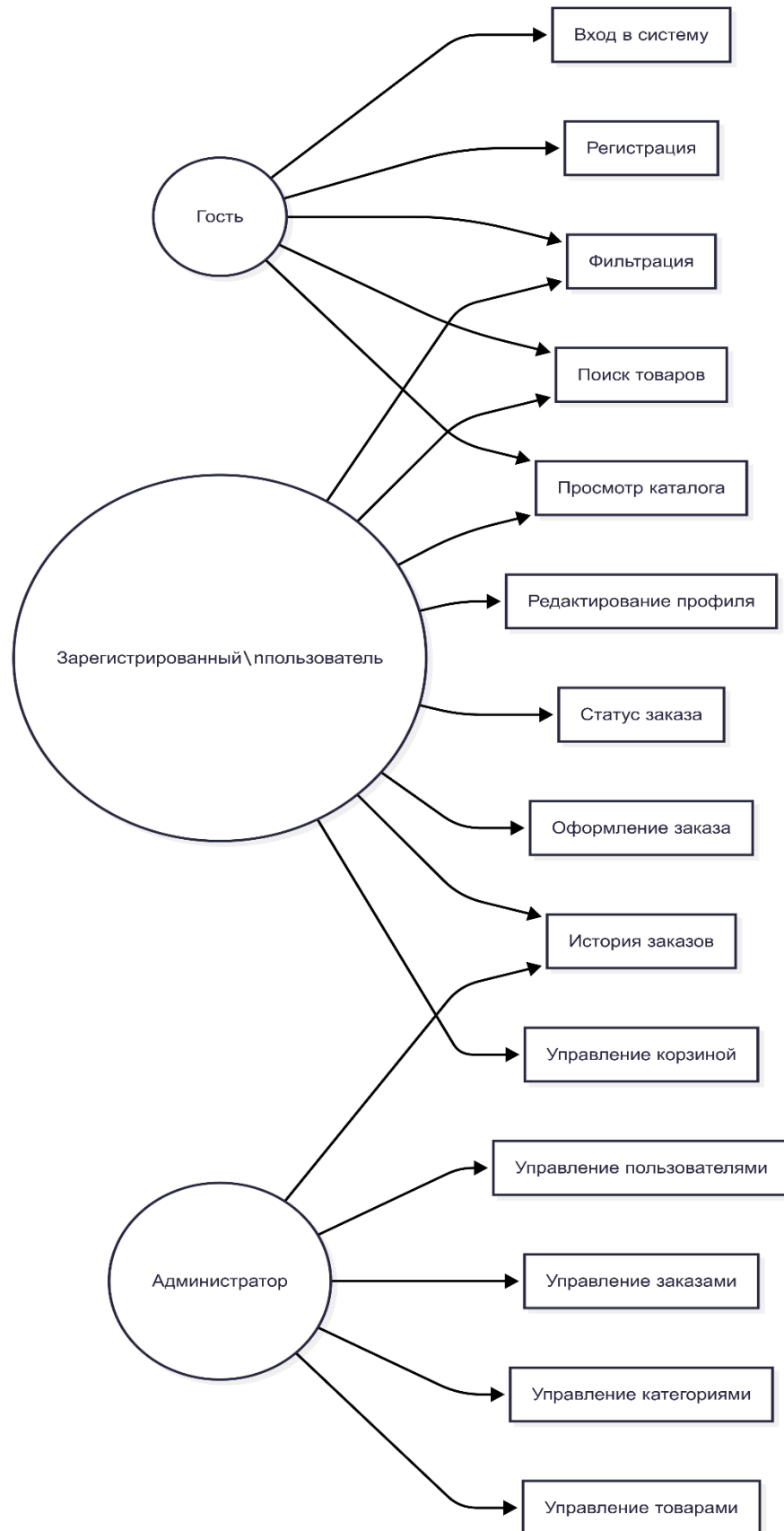
Для формализации требований к разрабатываемому веб-сервису было выполнено моделирование бизнес-процессов с использованием нотации UML.

2.1.1 Диаграмма вариантов использования (Use Case Diagram)

На рисунке 2.1 представлена диаграмма вариантов использования, отражающая функциональные возможности системы с точки зрения внешних акторов. В системе выделены три типа акторов:

- Гость (неавторизованный пользователь): имеет доступ к каталогу, поиску и страницам аутентификации.
- Зарегистрированный пользователь: обладает полным доступом к функционалу корзины, оформлению заказов, истории покупок и управлению профилем.
- Администратор: управляет каталогом товаров (CRUD) и отслеживает/изменяет статусы заказов.

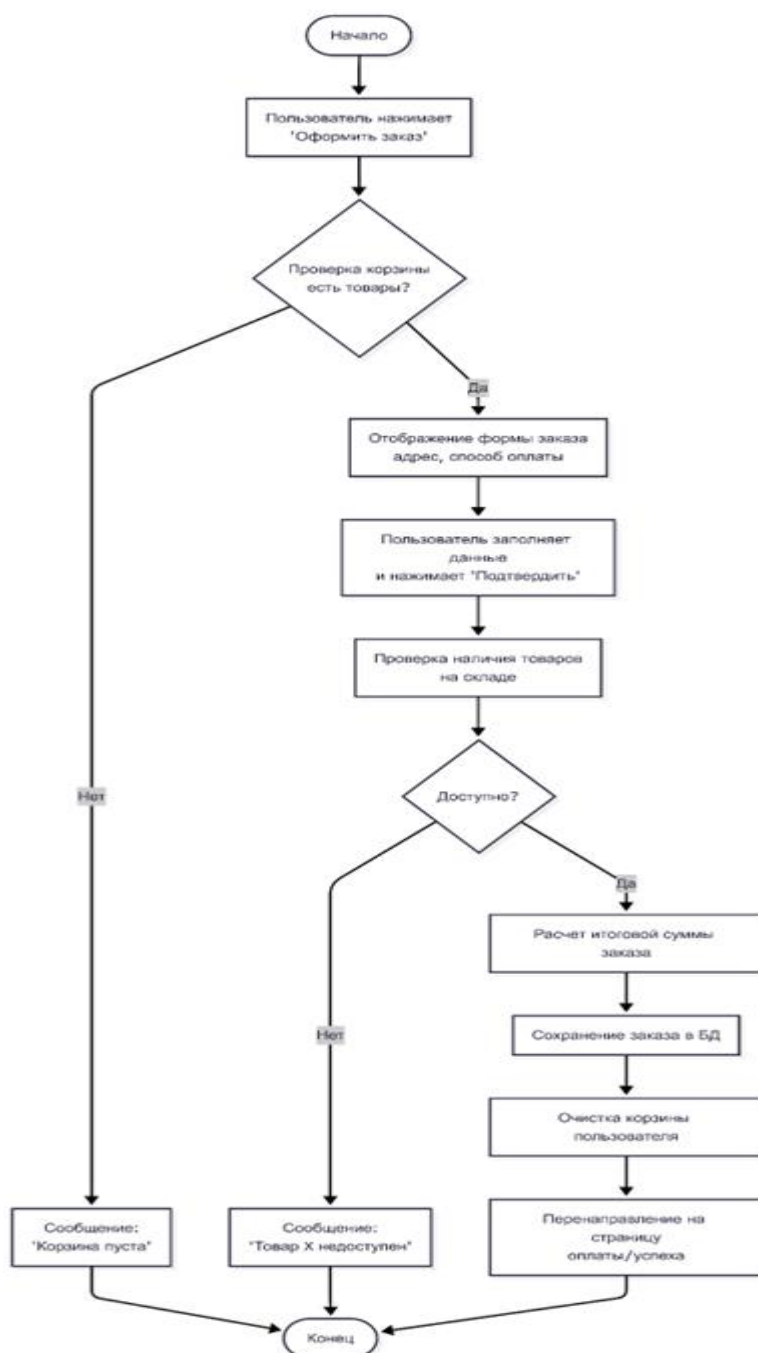
Рисунок 2.1 — Use Case Diagram



2.1.2 Диаграмма активности (Activity Diagram)

Ключевым бизнес-процессом является "Оформление заказа". На рисунке 2.2 представлена диаграмма активности, детализирующая этот процесс. Алгоритм включает проверку корзины, расчет итоговой стоимости (включая интеграцию с сервисами каталога и корзины), ввод адреса и выбор способа оплаты.

Рисунок 2.2 — Activity Diagram



2.2 Проектирование архитектуры веб-сервиса

2.2.1 Обоснование выбора микросервисной архитектуры

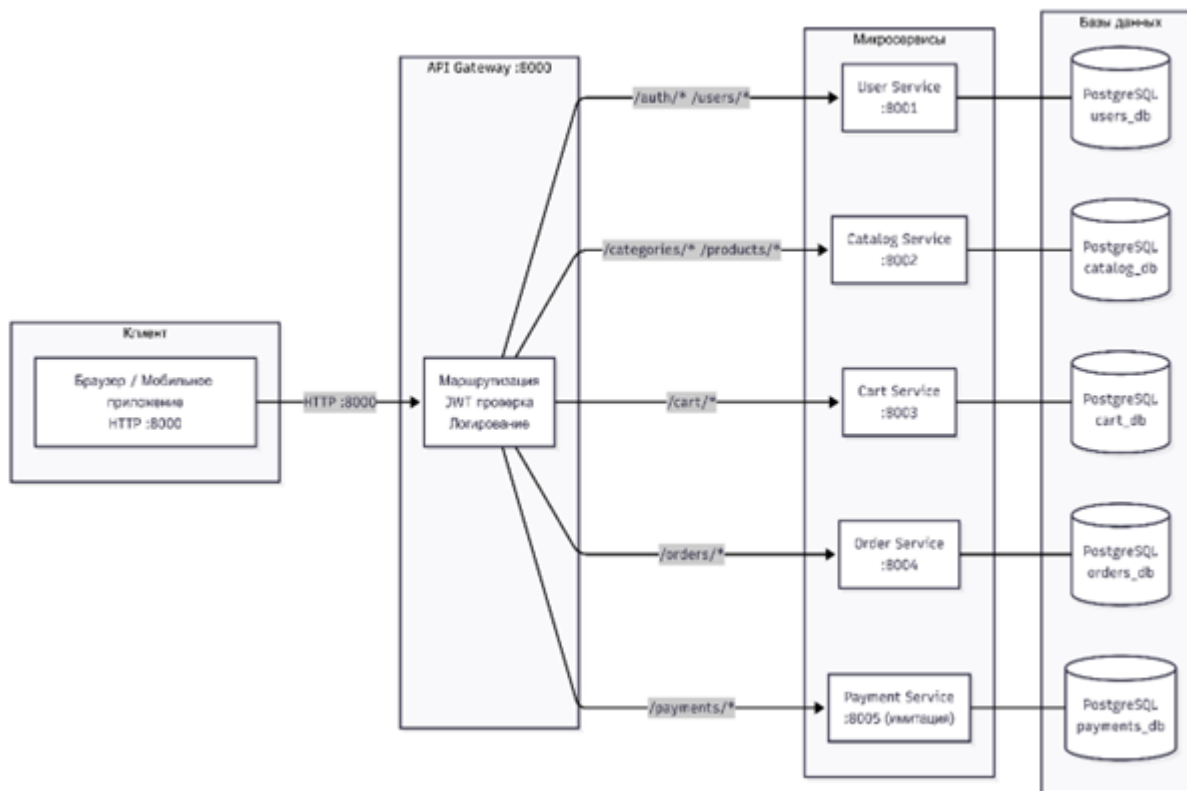
Проведённый в первой главе анализ требований к масштабируемости и гибкости системы позволил обосновать выбор микросервисной архитектуры. Данный подход обеспечивает слабую связанность компонентов, возможность независимого масштабирования наиболее нагруженных модулей (например, Catalog Service при сезонных распродажах) и упрощает внесение изменений в отдельные части системы без остановки всего приложения.

2.2.2 Состав микросервисов и их взаимодействие

Архитектура разрабатываемого решения включает следующие микросервисы:

1. User Service (Аутентификация и профили): выдача JWT-токенов, хранение данных пользователей.
2. Catalog Service (Каталог): управление товарами и категориями.
3. Cart Service (Корзина): временное хранение позиций, выбранных пользователем.
4. Order Service (Заказы): оркестрация процесса создания заказа, расчет финальной стоимости, управление статусами.
5. API Gateway (Единая точка входа): маршрутизация запросов, проверка JWT, агрегация данных.

Рисунок 2.3 — Схема взаимодействия



2.3 Проектирование базы данных

В соответствии с принципами микросервисной архитектуры, каждый сервис имеет собственную реляционную базу данных на базе СУБД PostgreSQL. Такой подход исключает связку сервисов через общие таблицы.

2.3.1 Логическая модель данных

- На рисунках 2.4-2.7 представлены ER-диаграммы для каждого сервиса.

- User Service: таблица users (id, email, password_hash, name, phone, role).
- Catalog Service: таблицы categories (id, name) и products (id, category_id, name, price, stock).
- Cart Service: таблица cart_items (id, user_id, product_id, quantity).
- Order Service: таблицы orders (id, user_id, total, status, address) и order_items (id, order_id, product_name, price, quantity).

Рисунок 2.4 — Таблица users service

users		
int	id	PK
string	email	UK
string	password_hash	
string	name	
string	phone	
string	address	
string	role	
datetime	created_at	

Рисунок 2.5 — Таблица categories service

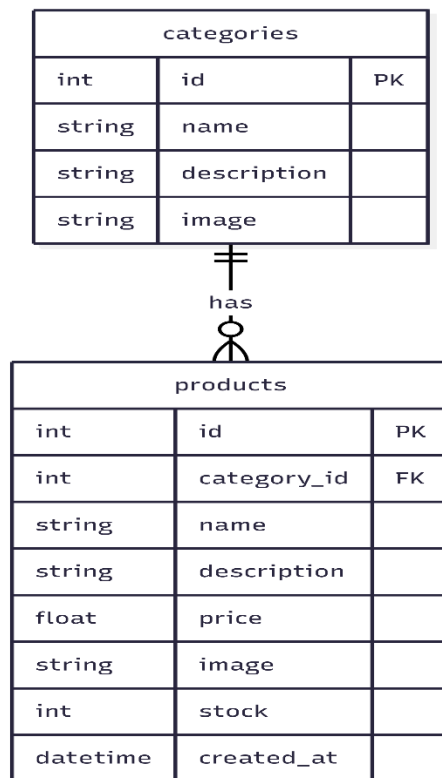
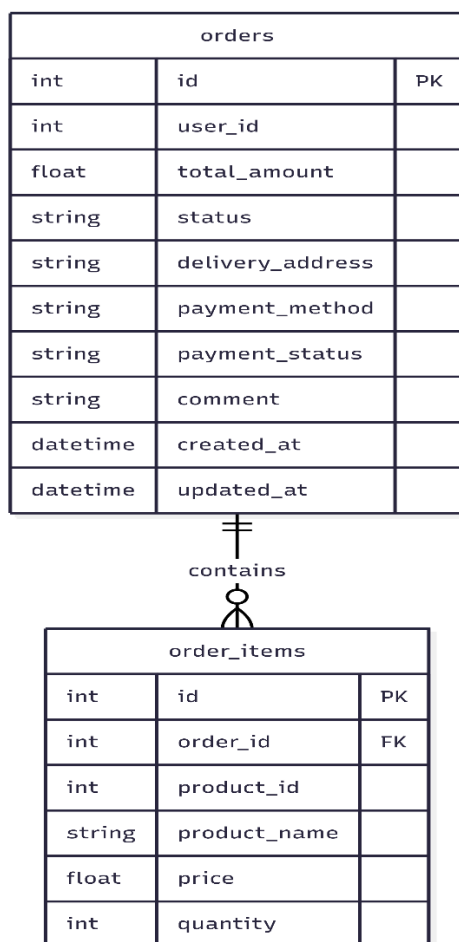


Рисунок 2.6 — Таблица cart service

cart_items		
int	id	PK
int	user_id	
int	product_id	
int	quantity	
datetime	created_at	

Рисунок 2.7 — Таблица orders service



2.4 Проектирование пользовательского интерфейса

При проектировании пользовательского интерфейса (UI) основное внимание уделялось интуитивности и адаптивности. На рисунках 2.8–2.16 представлены макеты ключевых экранов системы, выполненные в веб-интерфейсе.

Рисунок 2.8 — Макет главной страницы

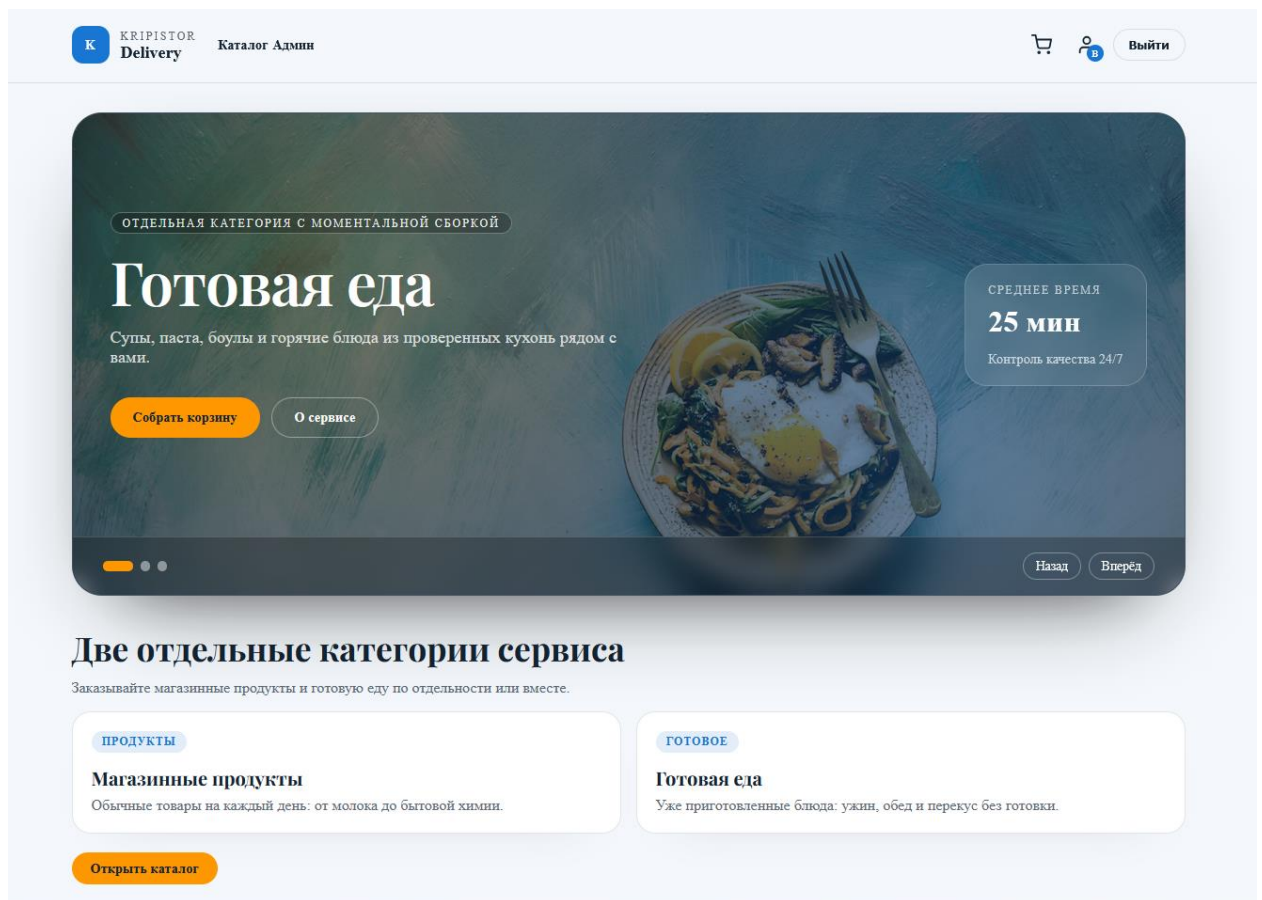


Рисунок 2.9 — Макет каталога

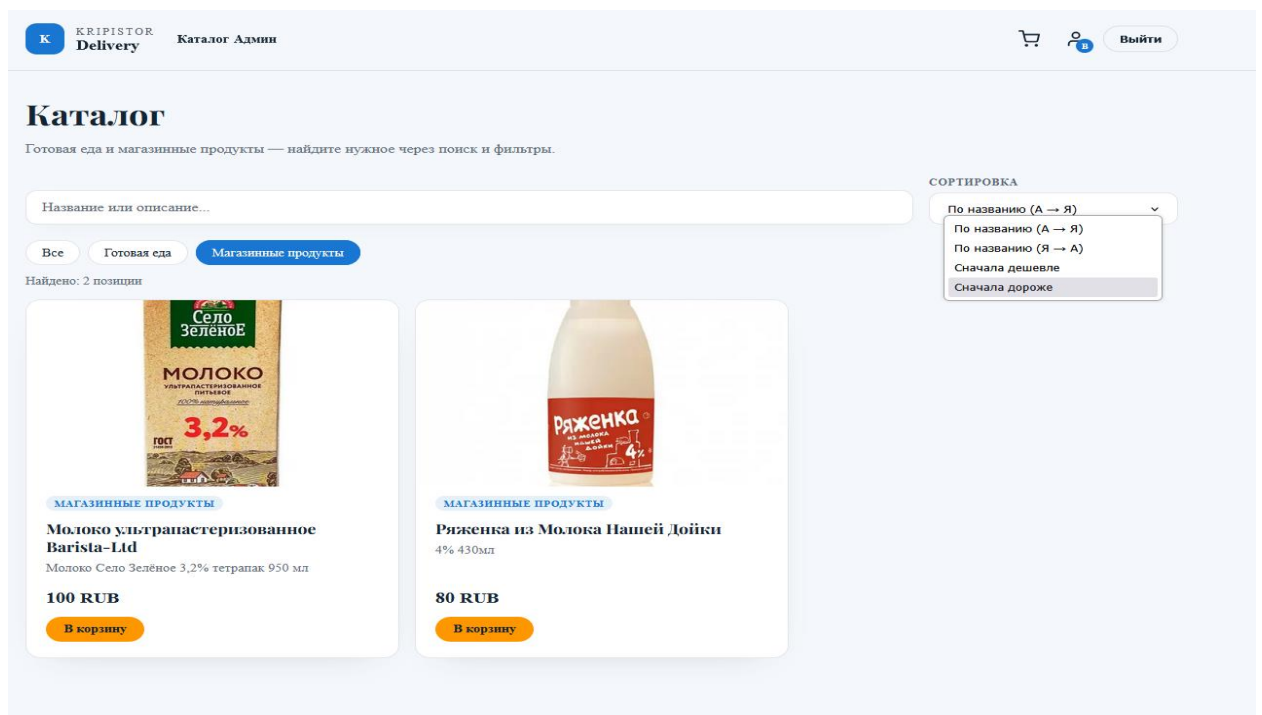


Рисунок 2.10 — Макет корзины

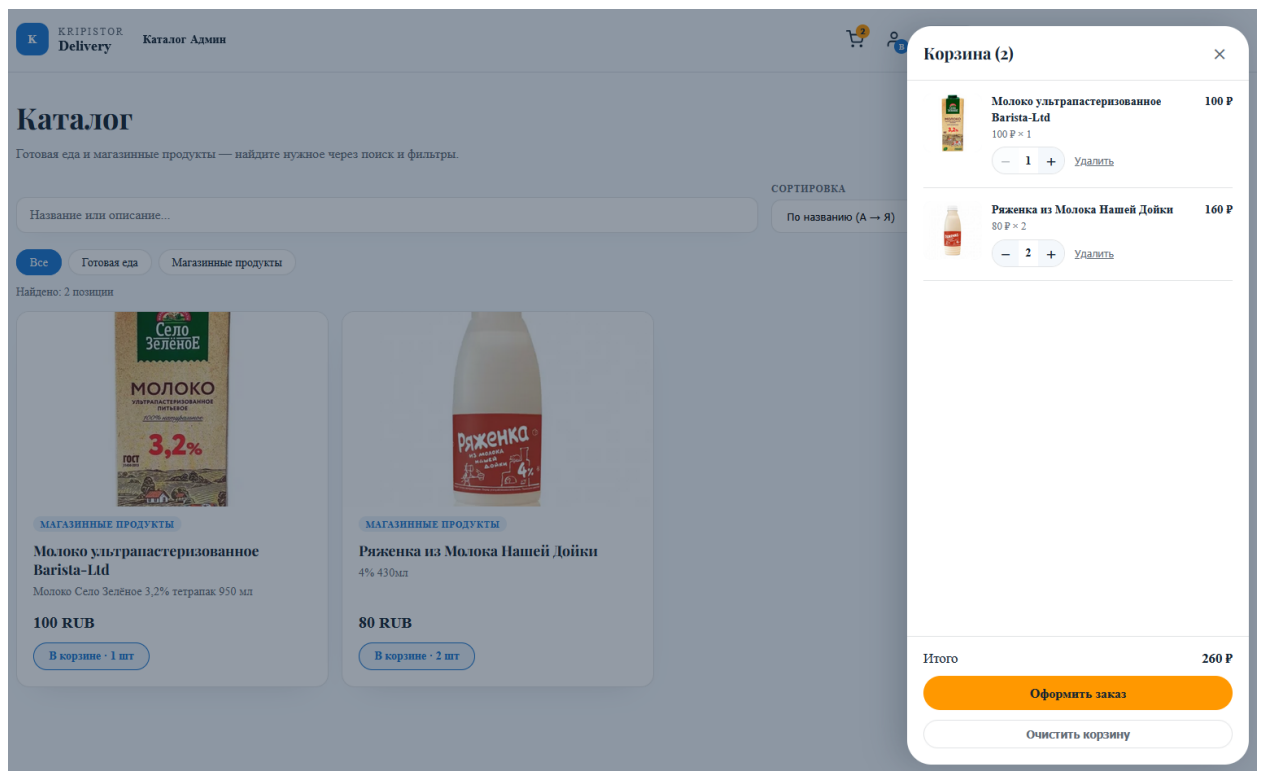


Рисунок 2.11 — Макет оформления заказа

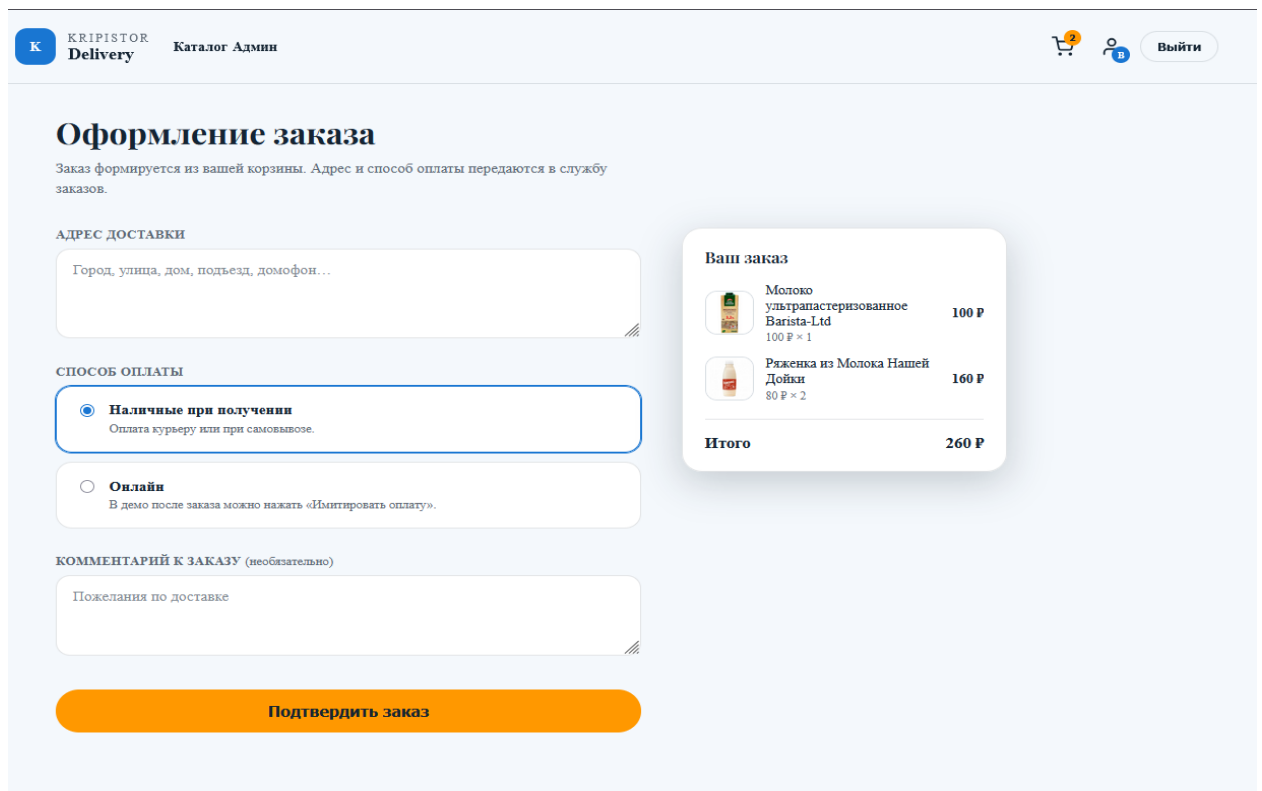


Рисунок 2.12 — Макет личного кабинета

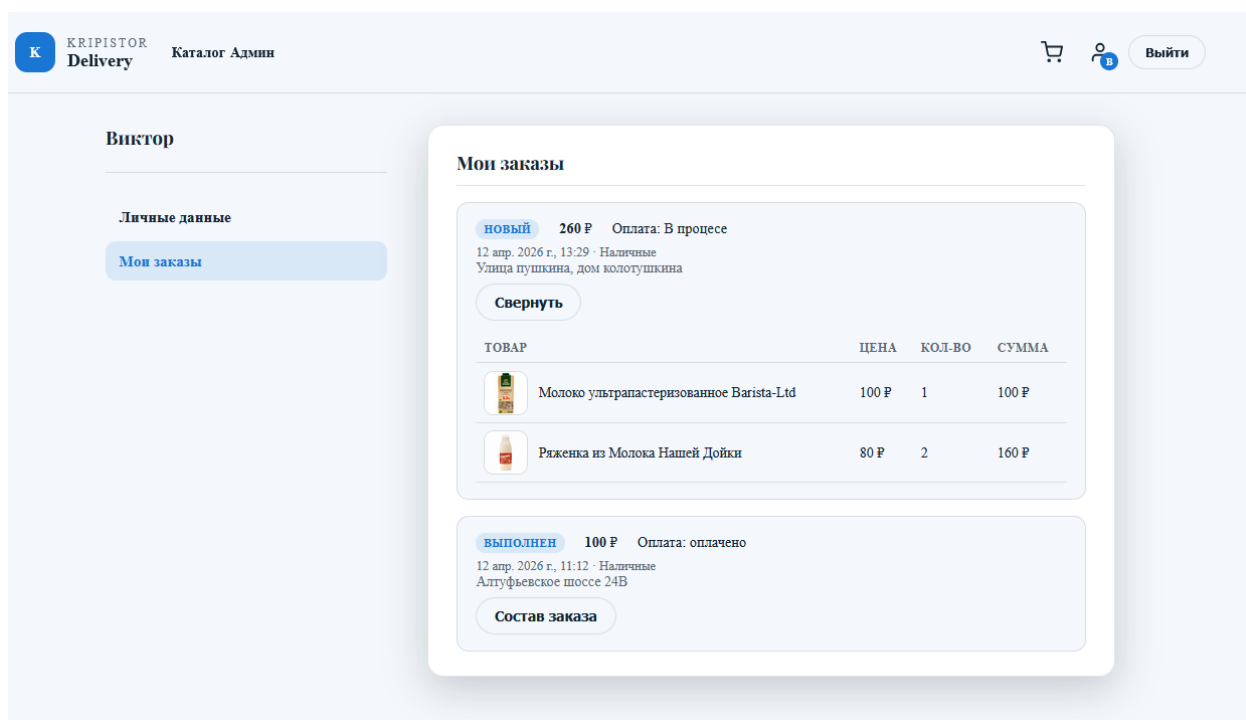


Рисунок 2.13 — Макет регистрации пользователя

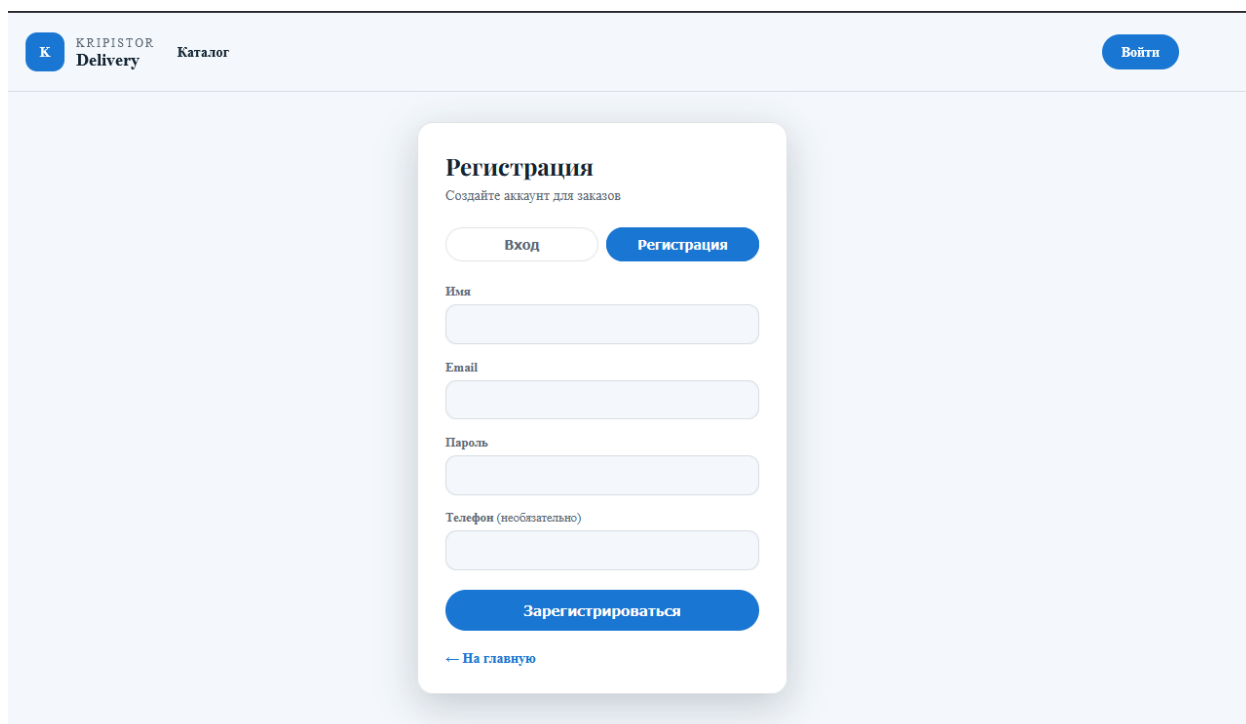


Рисунок 2.14 — Макет авторизации пользователя

К

KRIPISTOR
Delivery

Каталог

Войти

Вход

Регистрация

Войдите по email и паролю

Email

Пароль

Войти

[← На главную](#)

Рисунок 2.15 — Макет панели администратора

К

KRIPISTOR
Delivery

Каталог Админ

Выйти

Админ-панель

Категории

Товары

Заказы

Обновить каталог

Категории

Новая категория

НАЗВАНИЕ

ОПИСАНИЕ

ИЗОБРАЖЕНИЕ (ПО ЖЕЛАНИЮ)

Обзор... Файл не выбран.

Добавить категорию

ФОТО	НАЗВАНИЕ	ОПИСАНИЕ	ДЕЙСТВИЯ
	Молочные продукты	Лучшие молочные продукты	Редактировать

Рисунок 2.16 — Макет страницы оплаты

К KRIPISTOR Delivery Каталог Админ

Выйти

Оплата заказа

Реквизиты карты

Номер карты

4111 1111 1111 1111

Имя владельца

DEMO USER

Срок

12/30

CVC

123

Оплатить

Заказ

Номер: bab64341-e698-4b1b-ab14-2079f6a5c375

Сумма: 500 P

Статус оплаты: ожидает оплаты

[Перейти к моим заказам](#)

2.4.1 Описание пользовательских сценариев

Разработанный пользовательский интерфейс обеспечивает выполнение следующих ключевых сценариев работы с системой.

Сценарий 1 — Просмотр каталога товаров. Неавторизованный пользователь попадает на главную страницу, где отображаются категории товаров и популярные позиции. Пользователь может выбрать категорию для фильтрации товаров или воспользоваться поиском по названию. Для удобства навигации реализована пагинация — вывод товаров порциями по 12 штук на страницу.

Сценарий 2 — Добавление товаров в корзину. При нажатии на кнопку «В корзину» на карточке товара открывается модальное окно для выбора

количества (от 1 до 99). После подтверждения товар добавляется в корзину, а в иконке корзины в шапке сайта отображается актуальное количество позиций. Пользователь может перейти на страницу корзины, где видит список всех добавленных товаров с ценами и количеством, может изменить количество или удалить позицию.

Сценарий 3 — Оформление заказа. На странице корзины пользователь нажимает кнопку «Оформить заказ» и переходит к форме, где указывает адрес доставки (город, улица, дом, квартира, подъезд, домофон) и оставляет комментарий для курьера. Также пользователь выбирает способ оплаты: онлайн банковской картой или наличными при получении. После подтверждения заказа система проверяет наличие товаров на складе, резервирует их, создает запись в базе данных и очищает корзину.

Сценарий 4 — Управление заказами (администратор). Администратор авторизуется в системе и получает доступ к панели управления, где может просматривать список всех заказов с возможностью фильтрации по статусу (новые, в обработке, доставляются, завершённые, отменённые). Для каждого заказа администратор может изменить статус, что автоматически уведомляет пользователя (в интерфейсе).

Сценарий 5 — Просмотр истории заказов. Зарегистрированный пользователь в личном кабинете может просмотреть историю своих заказов: дату создания, общую сумму, статус выполнения. При нажатии на конкретный заказ открывается детальная информация с перечнем товаров, ценой каждого и итоговой суммой.

2.5 Проектирование алгоритмов работы системы

Для обеспечения корректной работы распределенной системы были формализованы ключевые алгоритмы.

2.5.1 Алгоритм регистрации и аутентификации

Процесс аутентификации реализован с использованием JWT-токенов.

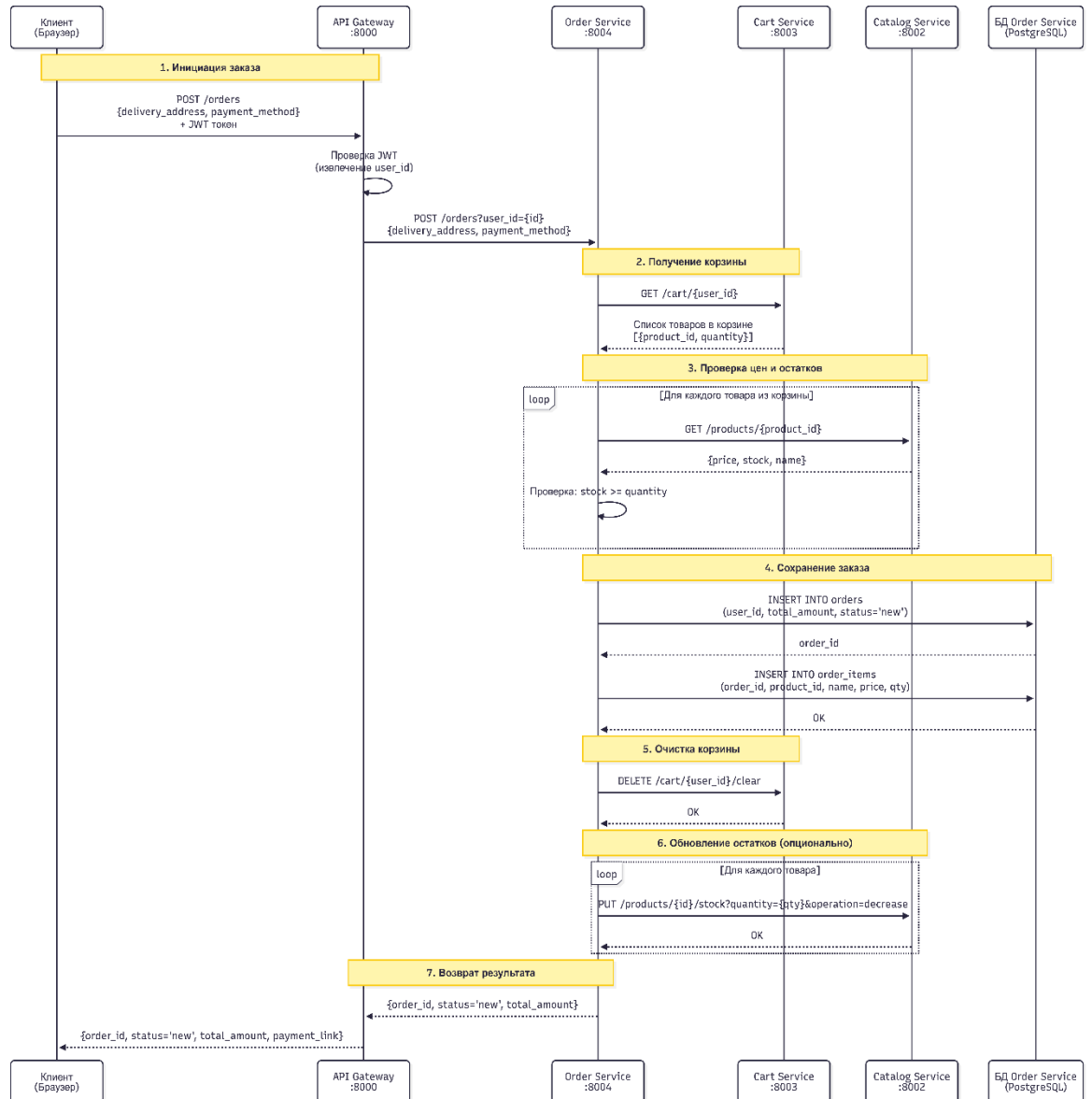
1. Клиент отправляет email и password на POST /auth/login в API Gateway.
2. Gateway проксирует запрос в User Service.
3. Сервис проверяет хеш пароля (bcrypt).
4. При успехе генерируется JWT, содержащий user_id и role, который возвращается клиенту.

2.5.2 Алгоритм взаимодействия микросервисов при оформлении заказа

Данный алгоритм является центральным в системе (Рисунок 2.17) и демонстрирует взаимодействие всех микросервисов:

1. Клиент -> API Gateway: POST /orders с адресом доставки и токеном в заголовке.
2. Gateway -> Order Service: Проверяет токен и передает запрос.
3. Order Service -> Cart Service: Запрашивает корзину пользователя (GET /cart/{user_id}).
4. Order Service -> Catalog Service: Для каждой позиции запрашивает актуальную цену и проверяет остаток (GET /products/{id}).
5. Order Service (Своя БД): Рассчитывает total_amount, сохраняет заказ в orders и детали в order_items.
6. Order Service -> Cart Service: Отправляет запрос на очистку корзины (DELETE /cart/{user_id}/clear).
7. Order Service -> Gateway: Возвращает клиенту JSON созданного заказа со статусом new и total_amount.

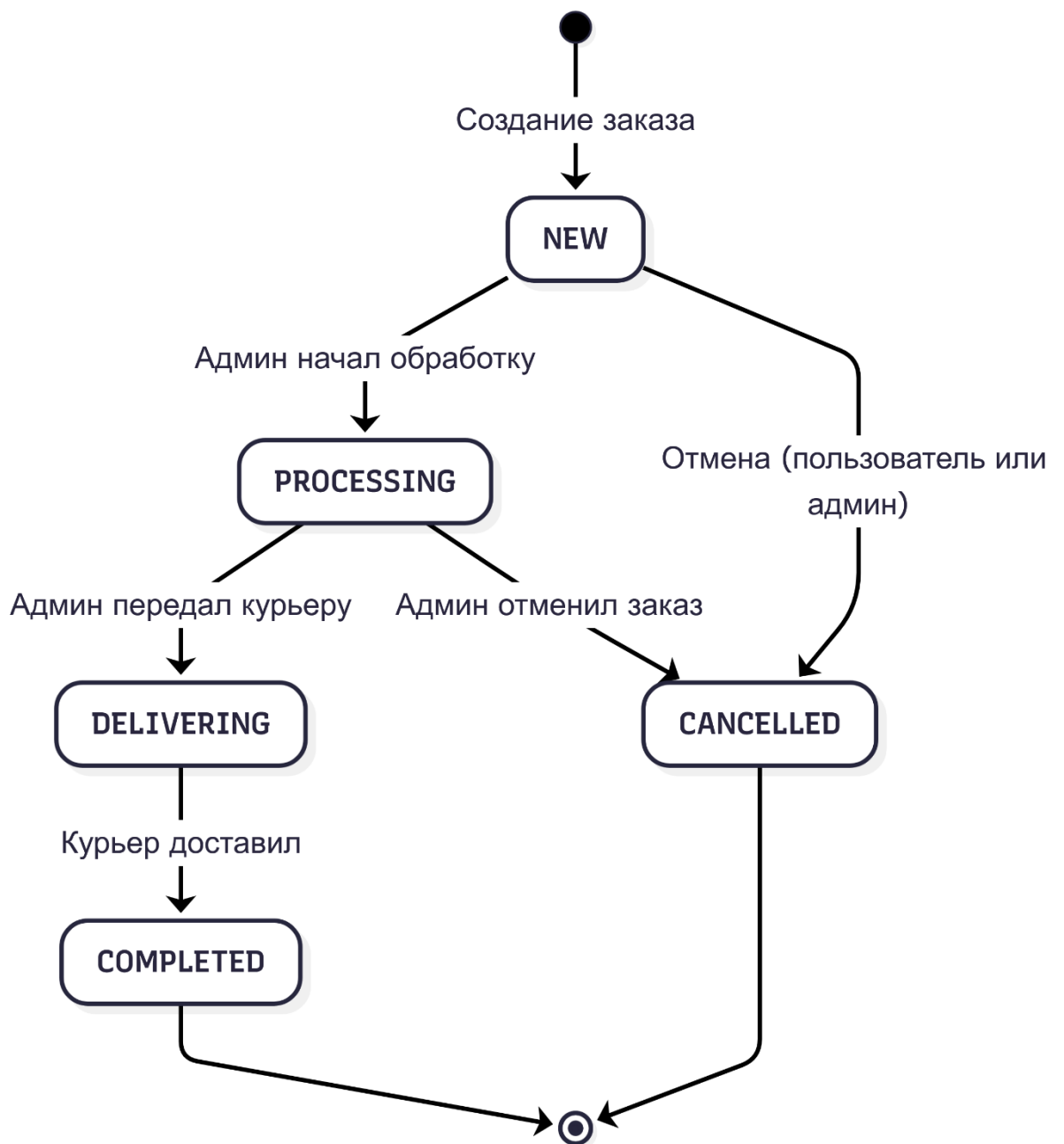
Рисунок 2.17 — Диаграмма последовательности оформления заказа



2.5.3 Алгоритм управления заказами (администрирование)

Администратор имеет возможность изменять статус заказа в процессе его выполнения. Статусная модель заказа реализована в виде конечного автомата, переходы между состояниями которого представлены на рисунке 2.18.

Рисунок 2.18 — Конечный автомат статусов заказа



Возможные статусы заказа:

NEW – заказ только что создан, администратор еще не начал обработку.

На этом этапе пользователь может отменить заказ самостоятельно.

PROCESSING – заказ принят в работу, идет сборка товаров. Отмена возможна только администратором.

DELIVERING – заказ передан курьеру и находится в пути. Изменение статуса возможно только администратором.

COMPLETED – заказ доставлен и закрыт. Финальный статус, дальнейшие изменения невозможны.

CANCELLED – заказ отменен. Финальный статус, дальнейшие изменения невозможны.

Допустимые переходы между статусами:

NEW → PROCESSING – администратор начинает обработку заказа.

NEW → CANCELLED – пользователь или администратор отменяет заказ.

PROCESSING → DELIVERING – администратор передает заказ курьеру.

PROCESSING → CANCELLED – администратор отменяет заказ.

DELIVERING → COMPLETED – курьер доставил заказ.

Недопустимые переходы (например, COMPLETED → PROCESSING) блокируются на уровне бизнес-логики. При попытке выполнить недопустимый переход сервер возвращает ошибку 400 Bad Request.

Каждое изменение статуса фиксируется в базе данных с отметкой времени. Это позволяет отслеживать историю выполнения заказа и анализировать скорость обработки.

3. РАЗРАБОТКА ВЕБ-СЕРВИСА ЗАКАЗА И ДОСТАВКИ ПРОДУКТОВ

3.1 Выбор средств разработки

При определении технологического стека для реализации веб-сервиса был проведен анализ доступных инструментов с учетом следующих критериев: производительность, масштабируемость, скорость разработки.

Для языка бэкенда рассматривались Python, Java и Go. Java требует больше кода для простых операций, Go менее распространен в веб-разработке. Python обеспечивает высокую скорость написания кода и имеет большое количество библиотек для веб-разработки и работы с базами данных. На основании этого для реализации бэкенда выбран Python 3.11.

Для веб-фреймворка рассматривались Django, Flask и FastAPI. Django признан избыточным для микросервисной архитектуры. Flask не поддерживает асинхронность и уступает в производительности. FastAPI является самым производительным, поддерживает асинхронные запросы и автоматически генерирует документацию Swagger UI. Выбран FastAPI.

В качестве СУБД рассматривались PostgreSQL и MySQL. Обе СУБД бесплатны. PostgreSQL обеспечивает лучшую поддержку ACID-транзакций, работу с JSON-данными и более строгое соответствие стандартам SQL. Выбрана PostgreSQL 15.

Для контейнеризации рассматривались Docker и Podman. Docker является более распространенным инструментом с большим количеством документации и примеров. Для оркестрации выбран Docker Compose, так как он позволяет одной командой запускать все микросервисы и базы данных.

Для языка фронтенда рассматривались JavaScript и TypeScript. TypeScript обеспечивает статическую типизацию, что позволяет выявлять ошибки на этапе компиляции. Выбран TypeScript.

Для фронтенд-фреймворка рассматривались стандартный React (SPA) и Next.js. Next.js обеспечивает встроенную поддержку серверного рендеринга (SSR), что улучшает SEO и скорость загрузки первой страницы, а также файловую маршрутизацию. Выбран Next.js 14

3.1.1 Детальное описание User Service

User Service реализует следующие функции: регистрация новых пользователей, аутентификация по email и паролю с выдачей JWT-токена, хранение профилей (имя, телефон, адрес), разделение ролей (пользователь / администратор). Для защиты паролей используется библиотека passlib с алгоритмом bcrypt, который автоматически генерирует соль и хеширует пароль перед сохранением в базу данных. JWT-токен содержит идентификатор пользователя, email и роль, что позволяет другим сервисам идентифицировать пользователя без повторных запросов к User Service. Срок действия токена установлен в 7 дней.

3.1.2 Детальное описание Catalog Service

Catalog Service управляет каталогом товаров и категорий. Реализованы CRUD-операции для товаров и категорий, поиск товаров по названию (регистронезависимый), фильтрация по категории и ценовому диапазону. Для оптимизации производительности все запросы к базе данных выполняются с использованием индексов по полям category_id и name. При создании заказа Order Service обращается к Catalog Service для проверки остатков товаров и получения актуальной цены (на момент заказа цена фиксируется в заказе, что позволяет изменять цены в каталоге без влияния на уже оформленные заказы).

3.1.3 Детальное описание Cart Service

Cart Service хранит временные данные о выбранных пользователем товарах. Корзина привязана к идентификатору пользователя и не требует авторизации для добавления товаров (гость может добавлять товары, но при оформлении заказа потребуются регистрация). При добавлении товара, который уже есть в корзине, увеличивается его количество. При изменении количества на 0 позиция удаляется. Данные корзины хранятся в PostgreSQL, однако при необходимости могут быть вынесены в Redis для повышения производительности.

3.1.4 Детальное описание Order Service

Order Service — центральный сервис системы, координирующий процесс оформления заказа. При поступлении запроса на создание заказа сервис выполняет следующие шаги: запрашивает корзину пользователя из Cart Service; для каждого товара запрашивает актуальную цену и проверяет остаток в Catalog Service; вычисляет общую сумму заказа; создает запись в таблице orders и записи в таблице order_items (с фиксацией цены на момент заказа); отправляет запрос на очистку корзины в Cart Service. Все операции с базой данных выполняются в рамках одной транзакции — при ошибке на любом этапе изменения откатываются.

3.1.5 Детальное описание API Gateway

API Gateway является единой точкой входа для всех клиентских запросов. Его основные функции: маршрутизация запросов к соответствующим микросервисам на основе префикса URL (например, /auth/* → User Service, /products/* → Catalog Service); проверка JWT-токенов для защищенных маршрутов (пропускаются только /auth/register и /auth/login); логирование всех входящих запросов для отладки. Gateway реализован на

FastAPI с использованием асинхронного HTTP-клиента `httpx` для проксирования запросов.

3.1.6 Обоснование выбора пользовательского интерфейса

При проектировании интерфейса учитывались следующие требования: интуитивная понятность, минимальное количество действий для оформления заказа, адаптивность под разные устройства.

Выбран минималистичный дизайн без отвлекающих элементов. Основные элементы управления (поиск, корзина, личный кабинет) вынесены в шапку сайта и доступны с любой страницы. Карточки товаров содержат только необходимую информацию: фото, название, цену и кнопку добавления в корзину. Такой подход ускоряет выбор товаров.

Процесс оформления заказа разбит на два шага: корзина и форма с адресом. Объединение в один экран перегружало бы интерфейс. Пользователь видит состав заказа до ввода адреса и может вернуться к редактированию.

Все интерактивные элементы имеют визуальную обратную связь: при наведении кнопка меняет цвет, при клике появляется состояние нажатия. При добавлении товара в корзину всплывает уведомление – пользователь понимает, что действие выполнено.

3.1.7 Требования к информационной безопасности

При разработке веб-сервиса учитывались базовые требования к информационной безопасности.

Аутентификация реализована с использованием JWT-токенов. Токен имеет срок действия 7 дней, после чего пользователь должен войти заново. Токен передается в заголовке `Authorization: Bearer <token>`. Это стандартный и безопасный подход для REST API.

Пароли пользователей не хранятся в открытом виде. Используется библиотека `passlib` с алгоритмом `bcrypt`. `Bcrypt` автоматически генерирует случайную «соль» для каждого пароля и имеет настраиваемый коэффициент сложности (`cost factor = 12`), что замедляет подбор паролей при возможной утечке базы данных.

Все запросы к API, кроме `/auth/register` и `/auth/login`, требуют наличия валидного JWT-токена. API Gateway проверяет токен на каждом запросе. При невалидном или просроченном токене сервер возвращает ошибку `401 Unauthorized`.

Базы данных каждого микросервиса изолированы друг от друга. Даже при компрометации одного сервиса злоумышленник не получит доступ к данным других сервисов. Подключение к базам данных происходит по внутренней сети Docker, порты баз данных не публикуются наружу.

Все микросервисы общаются между собой по HTTP внутри изолированной сети Docker. Публичный доступ есть только у API Gateway на порту 8000. Это минимизирует поверхность атаки.

3.2 Контейнеризация и оркестрация

Для упрощения развертывания все микросервисы упакованы в Docker-контейнеры. Docker Compose используется для одновременного запуска всех сервисов.

Листинг 3.1 – Dockerfile для User Service

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Листинг 3.2 – Фрагмент docker-compose.yml

```
services:
  user-service:
    build: ./services/user_service
    ports:
      - "8001:8000"
  catalog-service:
    build: ./services/catalog_service
    ports:
      - "8002:8000"
  cart-service:
    build: ./services/cart_service
    ports:
      - "8003:8000"
  order-service:
    build: ./services/order_service
    ports:
      - "8004:8000"
  api-gateway:
    build: ./services/api_gateway
    ports:
      - "8000:8000"
```

3.3 Реализация API Gateway

API Gateway является единой точкой входа. Он маршрутизирует запросы к соответствующим микросервисам и проверяет JWT-токены.

Листинг 3.3 – Маршрутизация запросов в API Gateway

```
SERVICE_ROUTES = {
    "/auth": "http://user-service:8000",
    "/users": "http://user-service:8000",
    "/products": "http://catalog-service:8000",
    "/cart": "http://cart-service:8000",
    "/orders": "http://order-service:8000",
}

@app.middleware("http")
async def gateway_middleware(request: Request, call_next):
    for prefix, url in SERVICE_ROUTES.items():
        if request.url.path.startswith(prefix):
            target_url = f"{url}{request.url.path}"
            async with httpx.AsyncClient() as client:
                response = await client.request(
                    method=request.method,
                    url=target_url,
                    headers=dict(request.headers),
                    content=await request.body()
                )
            return Response(content=response.content,
                             status_code=response.status_code)
    return await call_next(request)
```

3.4 Пример реализации микросервиса

Order Service взаимодействует с Cart Service и Catalog Service для получения корзины и проверки остатков товаров.

Листинг 3.4 – HTTP-клиент Order Service

```
async def get_cart(user_id: int):
    async with httpx.AsyncClient() as client:
        response = await
client.get(f"{CART_SERVICE_URL}/cart/{user_id}")
        return response.json()

async def get_product(product_id: int):
    async with httpx.AsyncClient() as client:
        response = await
client.get(f"{CATALOG_SERVICE_URL}/products/{product_id}")
        return response.json()

async def clear_cart(user_id: int):
    async with httpx.AsyncClient() as client:
        await client.delete(f"{CART_SERVICE_URL}/cart/{user_id}/clear")
```

3.5 Тестирование системы

3.5.1 Тест-кейсы

В таблице 3.1 представлены тест-кейсы для проверки API.

Таблица 3.1 – Тест-кейсы API

№	Тестируемый эндпоинт	Действие	Ожидаемый результат
1	POST /auth/register	Отправить email, пароль, имя	Код 200, JWT- токен
2	POST /auth/login	Отправить email, пароль	Код 200, JWT- токен
3	GET /products	Запрос списка товаров	Код 200, массив товаров
4	POST /cart/{id}/add	Добавить товар в корзину	Код 200, позиция в корзине
5	GET /cart/{id}	Получить корзину	Код 200, список товаров
6	POST /orders	Создать заказ	Код 200, заказ со статусом "new"

3.5.2 Результаты тестирования

Тестирование проводилось с использованием Swagger UI

Рисунок 3.1 — Регистрация пользователя

```
Curl  
  
curl -X 'POST' \  
    http://localhost:8001/auth/register' \  
    -H 'accept: application/json' \  
    -H 'Content-Type: application/json' \  
    -d '{  
      "email": "user@example.com",  
      "password": "string",  
      "name": "string",  
      "phone": "string",  
      "address": "string"  
    }'  
  
Request URL  
  
http://localhost:8001/auth/register  
  
Server response  
  
Code     Details  
  
200  
  
Response body  
  
{  
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 eyJzdMUiOlj1c2VyQGvYVWhtbwGUyZmFtLwduZXNlc19pc2l6ImR1bmV0ZDBhdGVpdHMtNDI3MC05MTI1NFY1YTlmNjcuaTI1ZmFtLnQvdGUiOi1j1c2VyIlwiZWhtIjoxNzc0NTkzMOTfOqfq.DgEIGIoZRQdz6U8zaftEt8Z0giFG8piCZhMspzf_Zkg",  
  "token_type": "bearer",  
  "user": {  
    "id": "de2f4dba-817c-4270-95b0-aea9f670126f",  
    "email": "user@example.com",  
    "name": "string",  
    "phone": "string",  
    "address": "string",  
    "role": "user",  
    "created_at": "2026-04-12T11:25:40.856750Z"  
  },  
}
```

Download

Рисунок 3.2 — Получение списка товаров

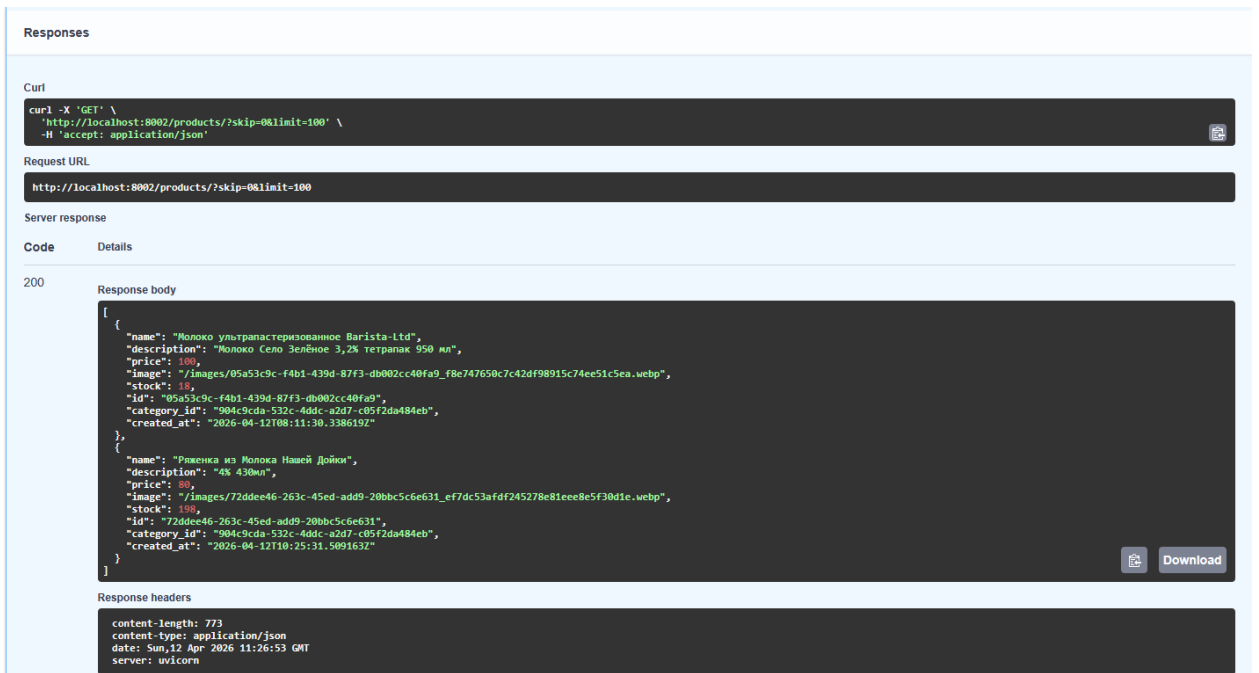
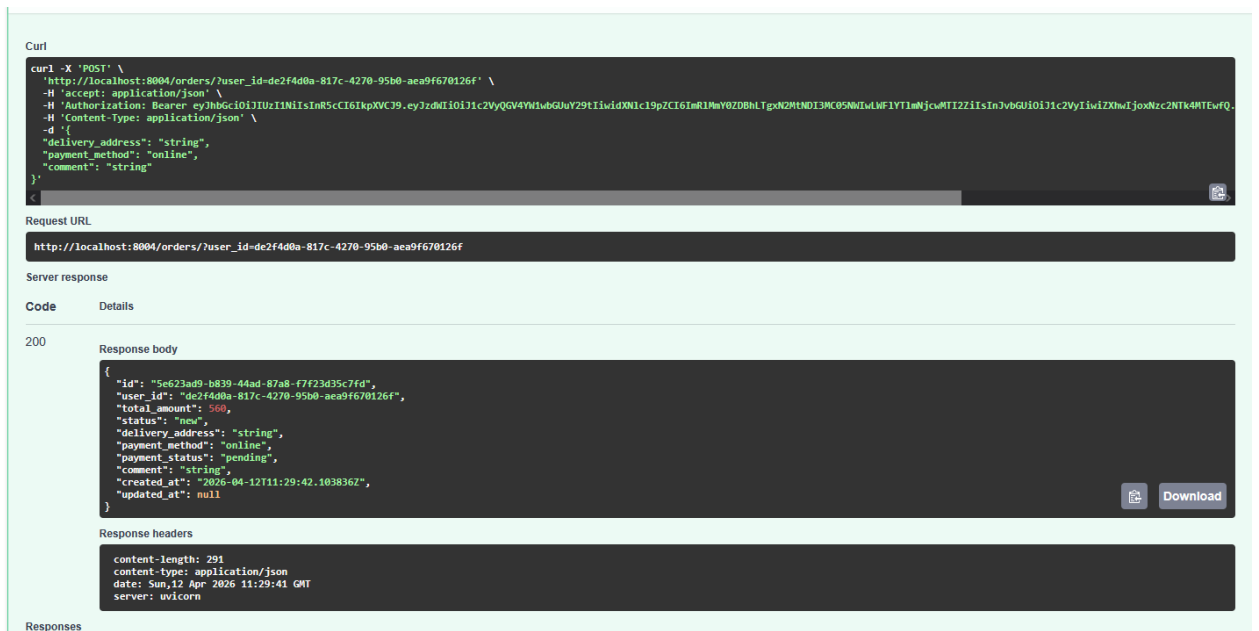


Рисунок 3.3 — Создание заказа



3.5.3 Проверка работоспособности

Для проверки работоспособности системы была выполнена команда `docker ps`, показывающая статус всех контейнеров.

Рисунок 3.4 — Статус контейнеров Docker

```
(.venv) PS D:\work\python\food_delivery> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
197e914f8888   food_delivery-api-gateway           "uvicorn app.main:ap..." 3 hours ago    Up 3 hours    0.0.0.0:8000->8000/tcp, [...]8000->8000/tcp  api-gateway
41d4eb21e995   food_delivery-order-service         "uvicorn app.main:ap..." 3 hours ago    Up 3 hours    0.0.0.0:8004->8000/tcp, [...]8004->8000/tcp  order-service
e5d9fcb5d539   food_delivery-cart-service          "uvicorn app.main:ap..." 3 hours ago    Up 3 hours    0.0.0.0:8003->8000/tcp, [...]8003->8000/tcp  cart-service
44c8e8131167   food_delivery-user-service          "uvicorn app.main:ap..." 3 hours ago    Up 3 hours    0.0.0.0:8001->8000/tcp, [...]8001->8000/tcp  user-service
452f5817a6d5   food_delivery-catalog-service       "uvicorn app.main:ap..." 3 hours ago    Up 3 hours    0.0.0.0:8002->8000/tcp, [...]8002->8000/tcp  catalog-service
b34889be4700   postgres:15                         "docker-entrypoint.s..." 3 hours ago    Up 3 hours    0.0.0.0:5432->5432/tcp, [...]5432->5432/tcp  user-db
4cb97d462d39   postgres:15                         "docker-entrypoint.s..." 3 hours ago    Up 3 hours    0.0.0.0:5435->5432/tcp, [...]5435->5432/tcp  order-db
515a82dd3007   postgres:15                         "docker-entrypoint.s..." 3 hours ago    Up 3 hours    0.0.0.0:5434->5432/tcp, [...]5434->5432/tcp  cart-db
b03d236639b7   postgres:15                         "docker-entrypoint.s..." 3 hours ago    Up 3 hours    0.0.0.0:5433->5432/tcp, [...]5433->5432/tcp  catalog-db
(.venv) PS D:\work\python\food_delivery>
```

3.5.4 Тестирование пользовательского интерфейса

Помимо тестирования API, было выполнено функциональное тестирование пользовательского интерфейса.

Таблица 3.2 – Тест-кейсы пользовательского интерфейса

№	Сценарий	Действие пользователя	Ожидаемый результат	Фактический результат
UI-01	Регистрация	Заполнить поля <i>email</i> , <i>пароль</i> , <i>имя</i> , <i>телефон</i> , нажать «Зарегистрироваться»	Перенаправление на главную страницу, в шапке отображается имя пользователя	Выполнено
UI-02	Вход	Ввести <i>email</i> и <i>пароль</i> , нажать «Войти»	Перенаправление на главную страницу, в шапке отображается имя пользователя	Выполнено
UI-03	Просмотр каталога	Нажать на категорию «Молочные продукты»	Отображаются только товары выбранной категории	Выполнено
UI-04	Поиск товаров	Ввести в поле поиска «молоко», нажать <i>Enter</i>	Отображаются товары, содержащие «молоко» в названии	Выполнено
UI-05	Добавление в корзину	На карточке товара выбрать количество 2, нажать «В корзину»	В иконке корзины отображается цифра 2	Выполнено

Продолжение Таблицы 3.2

UI-06	Изменение количества в корзине	На странице корзины изменить количество с 2 на 3	Сумма заказа пересчитана, отображается 3	Выполнено
UI-07	Удаление из корзины	Нажать иконку удаления рядом с товаром	Товар исчезает из списка, сумма обновляется	Выполнено
UI-08	Оформление заказа	Заполнить адрес, выбрать оплату онлайн, нажать «Подтвердить»	Перенаправление на страницу оплаты, заказ создан	Выполнено
UI-09	Просмотр истории заказов	В личном кабинете перейти в «История заказов»	Отображается список ранее оформленных заказов	Выполнено

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель — разработан веб-сервис заказа и доставки продуктов на основе микросервисной архитектуры.

В первой главе проведен анализ предметной области веб-сервисов заказа и доставки продуктов, в результате которого выявлены три основные бизнес-модели (агрегаторы, сервисы быстрой доставки, сервисы ритейлеров) и два архитектурных подхода (монолитный и микросервисный). Выполнен сравнительный анализ существующих сервисов (СберМаркет, Яндекс Лавка, Самокат) по критериям: бизнес-модель, ассортимент, скорость доставки, масштабируемость, стоимость внедрения. Обоснована необходимость разработки собственного сервиса, ориентированного на малый и средний бизнес, сформулированы требования: модульность, масштабируемость, функциональность.

Во второй главе разработаны модели бизнес-процессов в нотации UML: диаграмма вариантов использования, в которой выделены три актора (гость, пользователь, администратор), и диаграмма активности процесса оформления заказа. Спроектирована микросервисная архитектура, включающая пять компонентов: User Service, Catalog Service, Cart Service, Order Service и API Gateway. Обоснован выбор микросервисного подхода для обеспечения гибкости и масштабируемости. Разработана структура базы данных для каждого сервиса (таблицы users, categories, products, cart_items, orders, order_items) на СУБД PostgreSQL. Созданы макеты пользовательского интерфейса: главная страница, каталог, корзина, оформление заказа, личный кабинет, панель администратора, страницы регистрации и авторизации. Формализованы алгоритмы регистрации и аутентификации с использованием JWT-токенов и bcrypt, алгоритм оформления заказа, демонстрирующий взаимодействие всех микросервисов, а также алгоритм управления заказами.

В третьей главе выбран и обоснован стек технологий: для бэкенда – Python, FastAPI, PostgreSQL, SQLAlchemy, Docker; для фронтенда – TypeScript, Next.js, Tailwind CSS. Реализована контейнеризация всех микросервисов с использованием Docker Compose, что обеспечивает простоту развертывания и воспроизводимость окружения. Разработан API Gateway, выполняющий маршрутизацию запросов к соответствующим микросервисам и проверку JWT-токенов для защищенных маршрутов. Приведены листинги ключевых модулей: Dockerfile, docker-compose.yml, маршрутизация Gateway, HTTP-клиент Order Service. Проведено тестирование API с использованием Swagger UI, составлены тест-кейсы (регистрация, логин, получение товаров, добавление в корзину, создание заказа), все тесты пройдены успешно. Выполнена проверка работоспособности всех контейнеров через команду `docker ps`.

Практическая значимость работы заключается в создании готового к развертыванию веб-сервиса, который может быть использован малым и средним бизнесом для организации собственной службы доставки продуктов. В отличие от существующих коммерческих решений, разработанный сервис имеет открытую архитектуру, может быть адаптирован под конкретные нужды заказчика и не требует крупных инвестиций на старте.

Перспективы дальнейшего развития включают добавление сервиса оплаты с реальной интеграцией с платежными системами (ЮKassa или Stripe), внедрение очередей сообщений (RabbitMQ или Kafka) для асинхронного взаимодействия между сервисами, разработку мобильного приложения на React Native, добавление системы рекомендаций товаров на основе истории заказов, а также внедрение мониторинга (Prometheus + Grafana) и централизованного логирования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. E-commerce worldwide: Statista eCommerce Dossier / Altagamma, Statista [Электронный ресурс] // Altagamma : [сайт]. — URL: <https://altagamma.it/media/source/e-commerce-worldwide-statista-dossier.pdf> (дата обращения: 15.02.2026).
2. E-commerce в России: цифры, тренды и перспективы рынка / [Электронный ресурс] // Е-МБА : [сайт]. — URL: <https://e-mba.ru/knowledge-base/e-commerce-v-rossii-seychas-tsifry-trendy-i-perspektivy-rynka> (дата обращения: 18.02.2026).
3. Исследование рынка доставки еды в России / [Электронный ресурс] // DT-Team : [сайт]. — URL: <https://dt-team.ru/blog/ekonomika-i-biznes/issledovanie-rynka-dostavki-edy-v-rossii/> (дата обращения: 20.02.2026).
4. Newman S. Building Microservices: Designing Fine-Grained Systems. — 2nd ed. — O'Reilly Media, 2021. — 600 p.
5. Richardson C. Microservices Patterns: With examples in Java. — Manning Publications, 2018. — 520 p.
6. Fowler M., Lewis J. Microservices: a definition of this new architectural term [Электронный ресурс] // [MartinFowler.com](https://martinfowler.com) : [сайт]. — URL: <https://martinfowler.com/articles/microservices.html> (дата обращения: 25.02.2026).
7. Fielding R.T. Architectural Styles and the Design of Network-based Software Architectures. — Doctoral dissertation, University of California, Irvine, 2000. — 180 p.
8. Коммервилл И. Инженерия программного обеспечения. — 10-е изд. — М.: Вильямс, 2021. — 960 с.
9. Буч Г., Рамбо Д., Якобсон А. Язык UML. Руководство пользователя. — 2-е изд. — М.: ДМК Пресс, 2020. — 496 с.

- 10.Ларман К. Применение UML и шаблонов проектирования. — 3-е изд. — М.: Вильямс, 2019. — 736 с.
- 11.Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. — СПб.: Питер, 2020. — 448 с.
- 12.Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. — СПб.: Питер, 2021. — 352 с.
- 13.Холл К., Скотт-Флемминг К. FastAPI: современная разработка веб-API на Python. — М.: ДМК Пресс, 2023. — 280 с.
- 14.Lutz M. Learning Python. — 5th ed. — O'Reilly Media, 2018. — 1600 p.
- 15.Ramalho L. Fluent Python: Clear, Concise, and Effective Programming. — 2nd ed. — O'Reilly Media, 2022. — 1012 p.
- 16.PostgreSQL: The World's Most Advanced Open Source Relational Database [Электронный ресурс] // PostgreSQL Official Website : [сайт]. — URL: <https://www.postgresql.org> (дата обращения: 01.03.2026).
- 17.SQLAlchemy: The Python SQL Toolkit and Object Relational Mapper [Электронный ресурс] // [SQLAlchemy.org](https://www.sqlalchemy.org) : [сайт]. — URL: <https://www.sqlalchemy.org> (дата обращения: 03.03.2026).
- 18.Docker: Empowering App Development for Developers [Электронный ресурс] // Docker Official Website : [сайт]. — URL: <https://www.docker.com> (дата обращения: 05.03.2026).
- 19.Kubernetes: Production-Grade Container Orchestration [Электронный ресурс] // [Kubernetes.io](https://kubernetes.io) : [сайт]. — URL: <https://kubernetes.io> (дата обращения: 07.03.2026).

20. Next.js: The React Framework for the Web [Электронный ресурс] // Next.js Official Website : [сайт]. — URL: <https://nextjs.org> (дата обращения: 10.03.2026).
21. TypeScript: JavaScript with Syntax for Types [Электронный ресурс] // TypeScript Official Website : [сайт]. — URL: <https://www.typescriptlang.org> (дата обращения: 12.03.2026).
22. Tailwind CSS: Rapidly build modern websites without ever leaving your HTML [Электронный ресурс] // Tailwind CSS Official Website : [сайт]. — URL: <https://tailwindcss.com> (дата обращения: 14.03.2026).
23.  JWT: JSON Web Tokens for API Authentication [Электронный ресурс] // [JWT.io](https://jwt.io) : [сайт]. — URL: <https://jwt.io> (дата обращения: 17.03.2026).
24. Pydantic: Data validation using Python type hints [Электронный ресурс] // Pydantic Official Website : [сайт]. — URL: <https://docs.pydantic.dev> (дата обращения: 19.03.2026).
25. Uvicorn: An ASGI web server for Python [Электронный ресурс] // Uvicorn Official Website : [сайт]. — URL: <https://www.uvicorn.org> (дата обращения: 22.03.2026).
26. Яндекс Лавка: сервис заказа и доставки продуктов / [Электронный ресурс] // Yandex Dev : [сайт]. — URL: <https://dev.go.yandex/services/lavka> (дата обращения: 25.03.2026).
27. Самокат: быстрая доставка продуктов [Электронный ресурс] // Самокат : [сайт]. — URL: <https://samokat.ru> (дата обращения: 27.03.2026).
28. СберМаркет: доставка продуктов из магазинов [Электронный ресурс] // СберМаркет : [сайт]. — URL: <https://sbermarket.ru> (дата обращения: 29.03.2026).

- 29.ВкусВилл: интернет-магазин продуктов питания [Электронный ресурс] // ВкусВилл : [сайт]. — URL: <https://vkusvill.ru> (дата обращения: 31.03.2026).
- 30.Перекресток: онлайн-доставка продуктов [Электронный ресурс] // Перекресток : [сайт]. — URL: <https://perekrestok.ru> (дата обращения: 02.04.2026).
- 31.Ашан: интернет-магазин с доставкой [Электронный ресурс] // Ашан : [сайт]. — URL: <https://auchan.ru> (дата обращения: 04.04.2026).
- 32.Озон: маркетплейс с доставкой продуктов [Электронный ресурс] // Озон : [сайт]. — URL: <https://ozon.ru> (дата обращения: 06.04.2026).
- 33.Баринов С.Л. Системный анализ и моделирование бизнес-процессов. — М.: Финансы и статистика, 2020. — 320 с.
- 34.Гвоздева Т.В., Баллод Б.А. Проектирование информационных систем. — М.: Форум, 2021. — 400 с.
- 35.Чернышов Е.А. Разработка микросервисных приложений на Python. — СПб.: БХВ-Петербург, 2023. — 352 с.